

WIPRO NGA Program

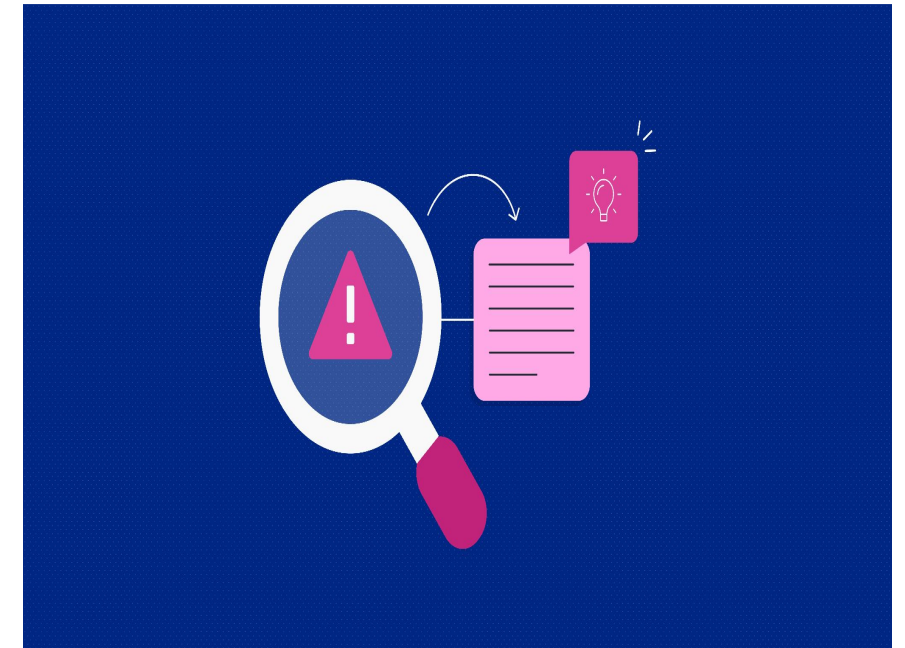
Capstone Project Presentation – 25th May, 2026

Project Title Here - Enterprise Concurrent Signal
Broadcast Framework (ECSBF) Implementation

Presented by - Saikat Sheet

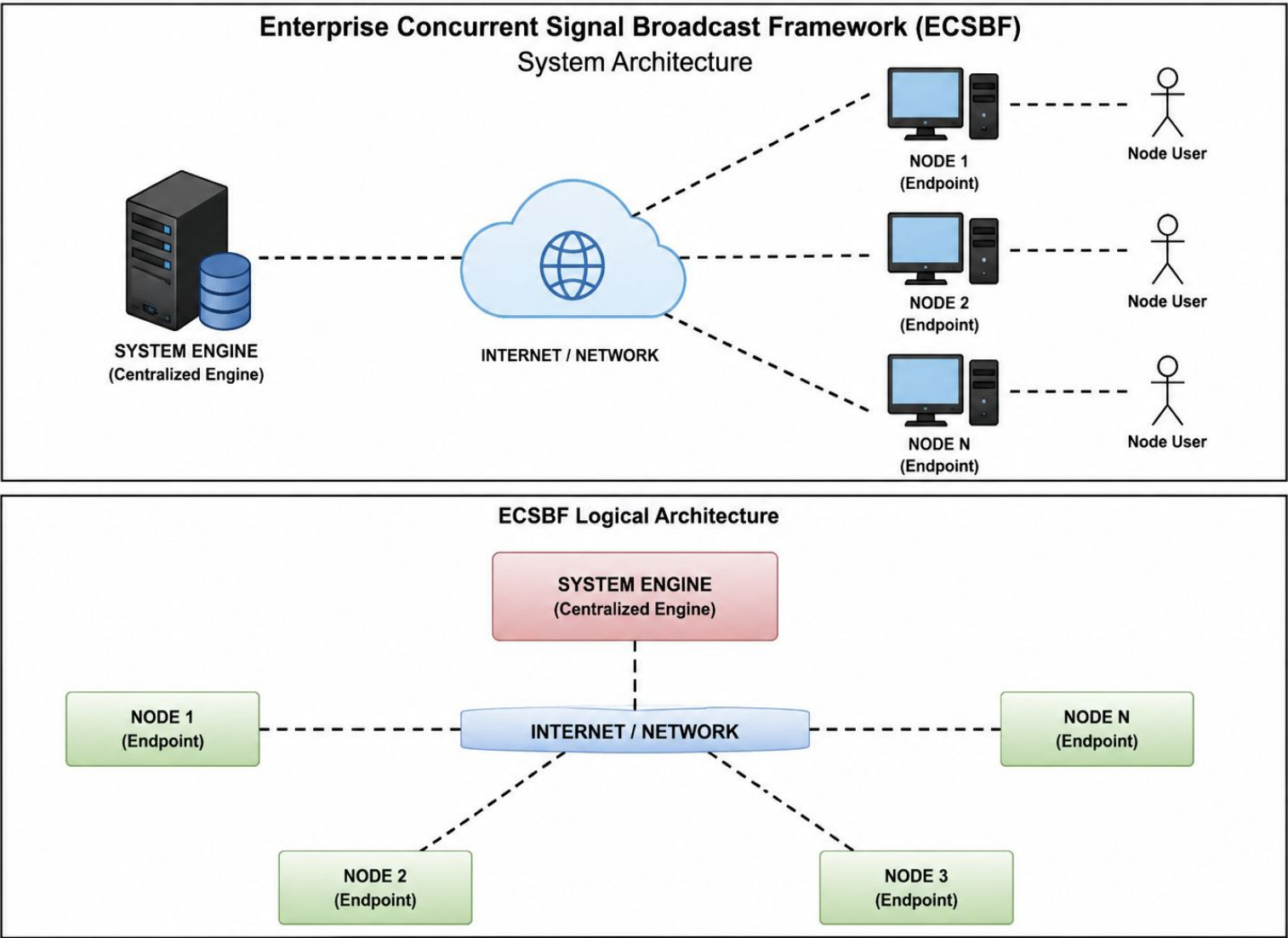
Enterprise Concurrent Signal Broadcast Framework (ECSBF) Implementation

- **A High-Speed Message Duplicator:** We need to build a central engine that can take an incoming message, copy it instantly, and send it to thousands of computers at the same time without any slowdowns.
- **A Reusable Code Framework:** We need to write the code in a modular, decoupled way so that this exact same engine can be dropped into any future project without rewriting its core code.
- **A Secure Gatekeeper:** We need to build a protection system at the network entry point that instantly blocks unauthorized or fake computers before they can waste the engine's processing power.



System Overview

The system engine acts as a central hub, broadcasting real-time data packets across the network to multiple endpoint nodes simultaneously.

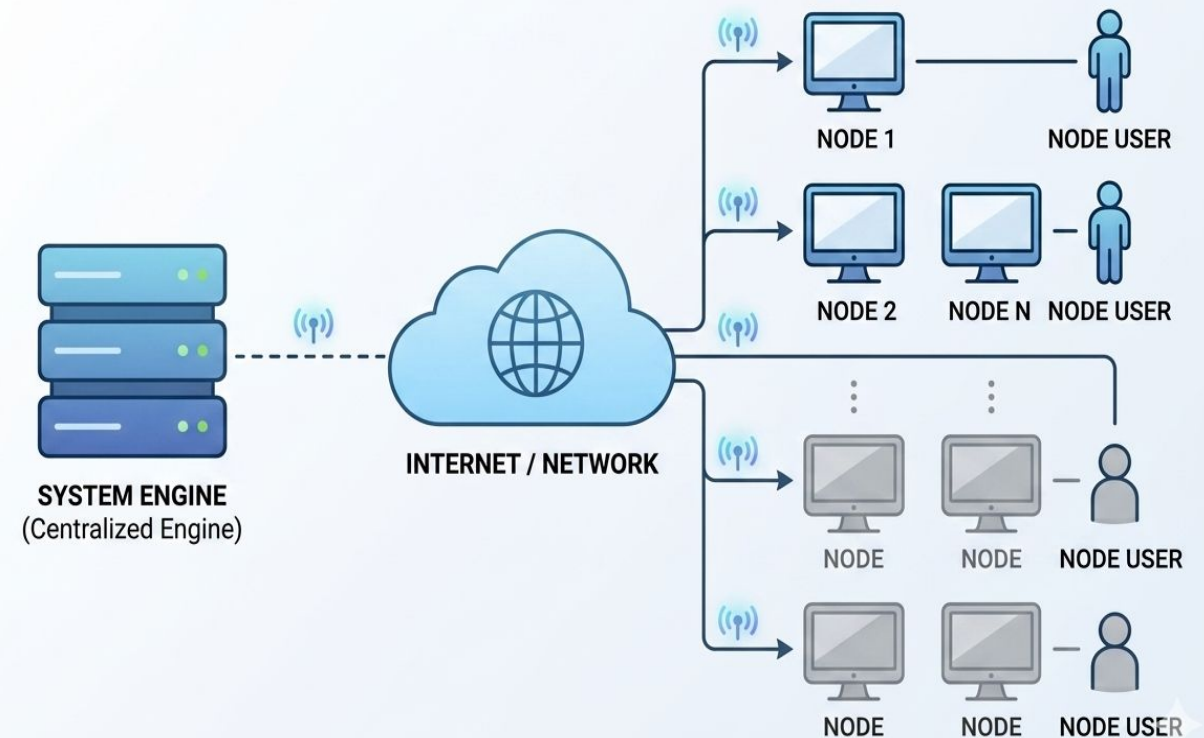


Concurrency Model

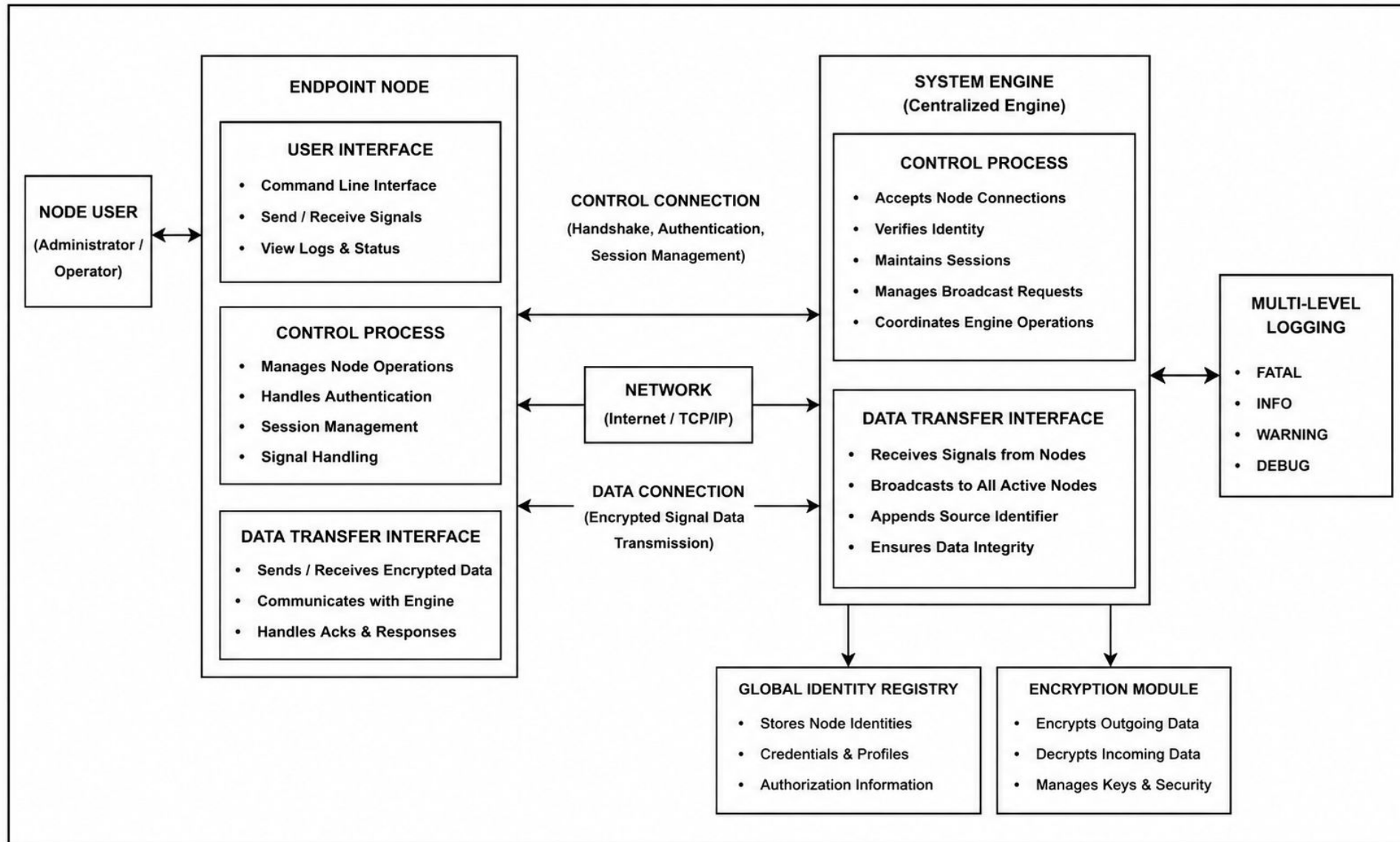
The system uses **thread pools** to process multiple signals simultaneously. To avoid race conditions and handle synchronization safely, it utilizes **mutexes (mutual exclusion)** and **atomic variables**. These lock critical data sections, ensuring only one thread updates connection states or log files at a time, preventing data corruption.

ENTERPRISE CONCURRENT SIGNAL BROADCAST FRAMEWORK (ECSBF)

A DECIDEDLY SIMPLIFIED OVERVIEW

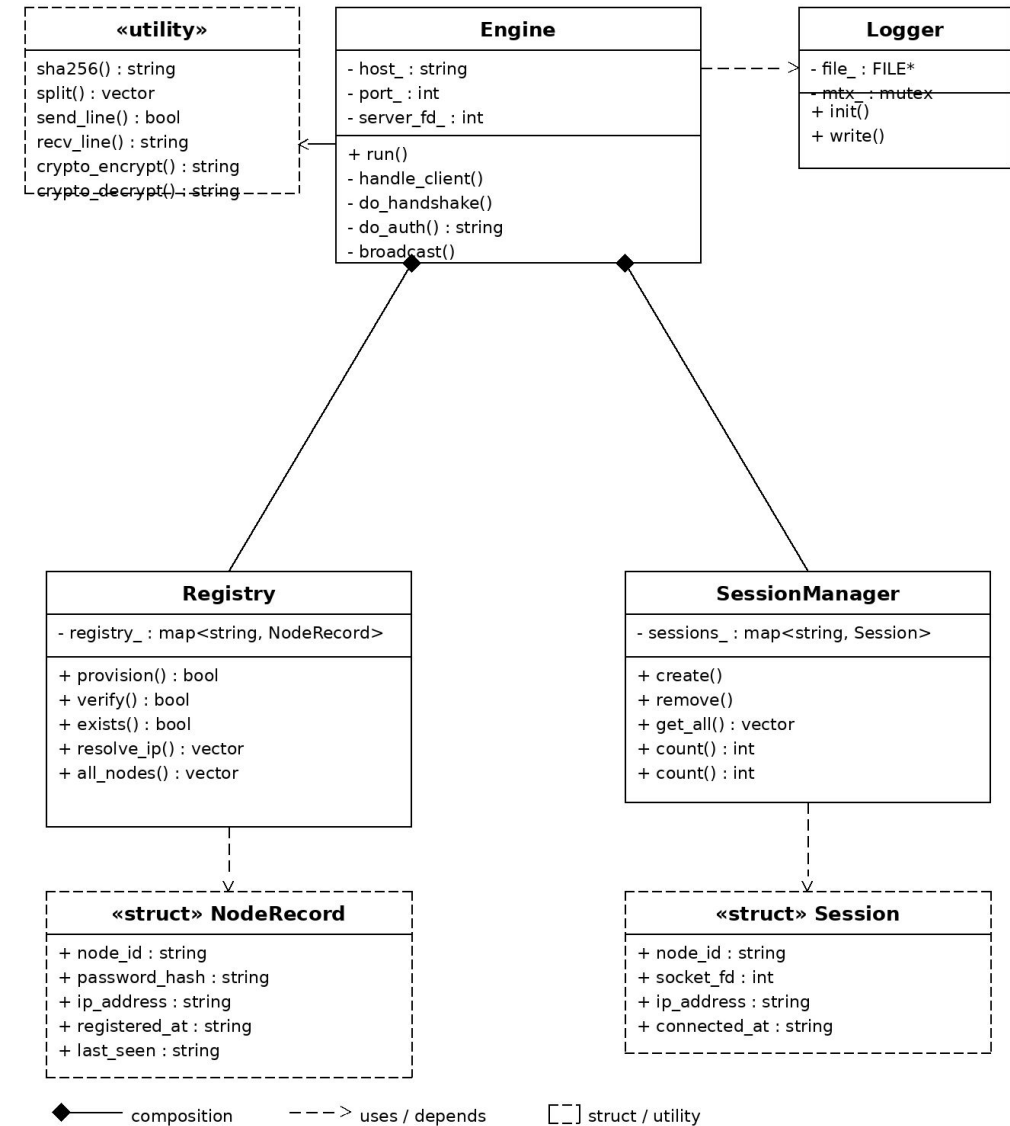


System Architecture



Class Diagram

- **Mediator Pattern:** The central Engine acts as the exclusive communication hub. Instead of components interacting directly, the engine orchestrates the entire workflow—handling client requests, validating security, and managing lookups safely in one spot.
- **Observer:** The framework handles dynamic data routing. The Registry ensures only verified nodes are allowed, while the SessionManager tracks live endpoints. The Engine::broadcast() method then pushes real-time signals to all active subscribers simultaneously.
- **Component :** The Engine maintains strict ownership (solid diamonds) over the Registry and SessionManager, managing their full life cycles directly.



Results

Engine

```
student@student: ~/25SUB5936_U21/24May26_SYS_CPPLinuxSysProg_CapStone_4026593-ECSBF/ECSBF
student@student: ~/25SUB5936... x student@student: ~/25SUB5936... x student@student: ~/Desktop/25... x student@student: ~/25SUB5936... x
student@student:~/25SUB5936_U21/24May26_SYS_CPPLinuxSysProg_CapStone_4026593-ECSBF/ECSBF$ ./bin/engine
[2026-05-23 18:55:09] [INFO ] === ECSBF Engine starting ===
[2026-05-23 18:55:09] [INFO ] Registry ready
[2026-05-23 18:55:09] [INFO ] SessionManager ready
[2026-05-23 18:55:09] [INFO ] Engine ready
[2026-05-23 18:55:09] [INFO ] Engine is online

ECSBF Engine - Online
Ready (Ctrl+C to stop)

[2026-05-23 18:55:34] [INFO ] Node registered: 'node_1'
[2026-05-23 18:55:34] [INFO ] Session started: 'node_1' (active: 1)
[+] 'node_1' connected (active: 1)
[>] 'node_1': Hello
[2026-05-23 18:55:43] [INFO ] 'node_1' broadcast: Hello
```

Node

```
student@student: ~/25SUB5936_U21/24May26_SYS_CPPLinuxSysProg_CapStone_4026593-ECSBF/ECSBF
student@student: ~/25SUB5936... x student@student: ~/25SUB5936... x student@student: ~/Desktop/25... x student@student: ~/25SUB5936... x
student@student:~/25SUB5936_U21/24May26_SYS_CPPLinuxSysProg_CapStone_4026593-ECSBF/ECSBF$ ./bin/node

ECSBF - Node Client

[1] Register (new account)
[2] Login (existing account)
[0] Exit

Select an option: 1

Node ID: node_1
Password: 123

[2026-05-23 18:55:34] [INFO ] Node 'node_1' starting
Registered:node_1

Type a message + Enter to send. Type quit to exit.

> Hello
[2026-05-23 18:55:43] [INFO ] 'node_1' sent a message
>
Message from 'node_1' =
Hello

> [2026-05-23 18:55:43] [INFO ] Received from 'node_1': Hello
```

Security & Reliability

- **Instant IP Blocking:** The system acts like a guard at the front gate, checking every computer trying to connect and instantly kicking out unknown or unauthorized ones.
- **Locked Data Packets:** All messages are completely encrypted from start to finish, meaning outsiders cannot read or mess with the data while it is being sent.
- **Protected Server Power:** By dropping bad connections right at the entryway, the system saves its processing power for real, authorized tasks.



Use Cases

Here are three practical, real-world scenarios where this broadcast framework is used:

1. Live Server Monitoring Systems

When a critical server in a massive data center suddenly overheats or runs out of memory, it sends out a warning signal. The broadcast framework instantly copies this alert and blasts it to thousands of dashboard screens, IT support computers, and automated backup systems at the exact same time so engineers can fix it before a website goes down.

2. Enterprise Emergency Alerts

In a large company with tens of thousands of employees, HR or security might need to send an urgent, company-wide safety notification. Instead of sending emails one by one which would take too long and clog the network the framework distributes the urgent message to every single active employee desktop app and communication channel simultaneously.

3. Distributed Workflow Triggers

When a customer places an order on a massive e-commerce website, that single event needs to trigger multiple actions at once. The framework immediately broadcasts the order signal to the inventory system to update stock, the billing system to process payment, and the shipping department to print a label, allowing all departments to start working in parallel.

Challenges

Project Hurdles & Solo Solutions

- **Meeting Timeline Goals**

- Formulated accurate milestone schedules and validated them through mentor reviews.
- Conducted structured self-evaluations to ensure steady progress between check-ins.
- Maintained a daily development log to stay disciplined and on track independently.

- **Controlling Project Scope**

- Co-created a clear feature boundary plan with my mentor to avoid adding unnecessary features.

- **Workflow Coordination**

- Selected the utility stack after consulting with my mentor on best practices.

- **System Engineering & Execution**

- Allocated strict personal time budgets for coding individual modules.
- Proactively pinpointed and fixed technical vulnerabilities based on mentor feedback early in the codebase.

Future Enhancements

- **Dynamic Load Balancing:** Automatically shifts heavy broadcasting traffic across backup servers during massive spikes.
- **Encrypted Storage:** Adds secure database logging to permanently store and audit all transmitted message details.
- **Web Dashboard:** Builds a real-time monitor interface to track active nodes and network health instantly.



Conclusion

The Enterprise Concurrent Signal Broadcast Framework successfully achieves high-speed, real-time message distribution. By combining asynchronous C++ networking, robust multithreading patterns, and strict socket-level security, the architecture delivers a reliable, scalable, and isolated communication hub capable of handling intensive enterprise workloads without network degradation.

*Thank
you!*